COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010/ Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	August 1, 2026	**DATE SUBMITTED:**	August 4, 2026
NAME:	EUNICE E. MORADILLO	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

ACTIVITY NO. 4 **Stacks**

1. PROCEDURE OUTPUT

ILO A

```cpp
#include <iostream>
#include <stack>

int main(){

std :: stack<int> stack1;
std :: cout << "\n --- STACK 1 --- " << std :: endl;
std :: cout << "Testing the stack STL" << std :: endl;

//push
std :: cout << "is the stack empty? " << stack1.empty() << std :: endl;

stack1.push(10);
std :: cout << "the top of the stack is " << stack1.top() << std :: endl;

stack1.push(9);
std :: cout << "the top of the stack is " << stack1.top() << std :: endl;

stack1.push(7);
std :: cout << "the top of the stack is " << stack1.top() << std :: endl;

stack1.push(4);
std :: cout << "the top of the stack is " << stack1.top() << std :: endl;

stack1.push(3);
std :: cout << "the top of the stack is " << stack1.top() << std :: endl;

stack1.push(2);
std :: cout << "the top of the stack is " << stack1.top() << std :: endl;

stack1.pop();

std :: cout << "the top of the stack is " << stack1.top() << std :: endl;

std :: cout << "is the stack empty? " << stack1.empty() << std :: endl;

std :: cout << "the size of the stack is " << stack1.size() << std :: endl;

std :: stack<int> stack2;
std :: cout << "\n ..- STACK 2 --- " << std :: endl;

stack2.emplace(5);
std :: cout << "The top of the stack 2 is: " << stack2.top() << std :: endl;

stack2.emplace(6);
std :: cout << "The top of the stack 2 is: " << stack2.top() << std :: endl;

stack2.emplace(1);
std :: cout << "The top of the stack 2 is: " << stack2.top() << std :: endl;

stack2.emplace(9);
std :: cout << "The top of the stack 2 is: " << stack2.top() << std :: endl;

std :: cout << "\nBefore swap:" << std :: endl;
std :: cout << "The top of the stack 1 is: " << stack1.top() << std :: endl;
std :: cout << "The top of the stack 2 is: " << stack2.top() << std :: endl;
stack1.swap(stack2);

std :: cout << "\nAfter swap:" << std :: endl;
std :: cout << "The top of the stack 1 is: " << stack1.top() << std :: endl;
std :: cout << "The top of the stack 2 is: " << stack2.top() << std :: endl;

}
```

```
 --- STACK 1 ---
Testing the stack STL
is the stack empty? 1
the top of the stack is 10
the top of the stack is 9
the top of the stack is 7
the top of the stack is 4
the top of the stack is 3
the top of the stack is 2
the top of the stack is 3
is the stack empty? 0
the size of the stack is 5

 ..- STACK 2 ---
The top of the stack 2 is: 5
The top of the stack 2 is: 6
The top of the stack 2 is: 1
The top of the stack 2 is: 9

Before swap:
The top of the stack 1 is: 3
The top of the stack 2 is: 9

After swap:
The top of the stack 1 is: 9
The top of the stack 2 is: 3
```

ANALYSIS:

In this code, using the stl stack is more simple than the others because it already has a built in function that can be easily called, like push, pop and emplace so i dont have to declare and write the code each. Testing stack1 showed that push adds values on top and pop removes the last one added, which shows the LIFO order. The empty() function correctly returned 1 when the stack still had nothing pushed, and 0 once it had values. I also tested emplace() on stack2, which works the same as push but constructs the element directly. The swap() function was useful because it let me swap the contents of stack1 and stack2 without manually looping through each element.

ILO B

GLOBAL DECLARATION

```
#include<iostream>

//global declaration
#define maxCap 10

int stackArr[maxCap];
int top = -1, newData;
//prototype functions
void push();
void pop();
void Top();
void displayStack();
bool isEmpty();
bool isFull();


//create a function for displayStack
void displayStack();
void exit();
```

MAIN

```
int main(){
//main driver
 int choice;
 while(true)
 {
    std::cout<< "================================================================================" << std::endl;
    std::cout<<"                                Stack Operations \n";
    std::cout<< "   1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit"<<std::endl;
    std::cout<< "--------------------------------------------------------------------------------" << std::endl;
    std::cin >> choice;

 switch(choice){
    case 1: push();
    break;
    case 2: pop();
    break;
    case 3 : Top();
    break;
    case 4: std::cout << "is the stack empty? " << isEmpty() <<std::endl;
    break;
    case 5: std::cout<< "is the stack full? "<< isFull() <<std::endl;
    break;
    case 6: displayStack();
    break;
    case 7: exit();
    break;
    default : std::cout << "Invalid choice" << std::endl;
    break;

    }
 }
}
```

FUNCTION DEFINITION

```
bool isEmpty(){
    //how do we verify if the stack is empty
    if (top == -1) return true;
    return false;
}
```

```
================================================================================
                                Stack Operations
   1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------------
4
is the stack empty? 0
```

<table>
<tr>
<td></td>
<td>

```
==================================================================
                    Stack Operations
  1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
------------------------------------------------------------------
4
is the stack empty? 1
```

</td>
</tr>
<tr>
<td>

```
bool isFull(){
    if(top == maxCap - 1) return true;
    return false;
}
```

</td>
<td>

```
==================================================================
                    Stack Operations
  1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
------------------------------------------------------------------
5
is the stack full? 1
```

```
==================================================================
                    Stack Operations
  1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
------------------------------------------------------------------
5
is the stack full? 0
==================================================================
```

</td>
</tr>
</table>

```
void push(){
    //error checking??
    if(isFull()){
        std::cout<< "Stack Overflow" << std::endl;
        return;
    }
    //pushing to the stack
    std::cout << "Enter a new Value: " << std::endl;
    std::cin >> newData;
    // how do we insert the data into the stack
    stackArr[++top] = newData;
}
```

```
  1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------------
1
Enter a new Value:
6
================================================================================
                        Stack Operations
  1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------------
1
Enter a new Value:
5
================================================================================
                        Stack Operations
  1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------------
1
Enter a new Value:
9
================================================================================
                        Stack Operations
  1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------------
1
Enter a new Value:
0
================================================================================
                        Stack Operations
  1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------------
1
Enter a new Value:
6
================================================================================
                        Stack Operations
  1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------------
1
Enter a new Value:
4
================================================================================
                        Stack Operations
  1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------------
1
Enter a new Value:
4
================================================================================
                        Stack Operations
  1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------------
1
Enter a new Value:
8
================================================================================
                        Stack Operations
  1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------------
1
Enter a new Value:
6
================================================================================
                        Stack Operations
  1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------------
1
Stack Overflow
================================================================================
                        Stack Operations
  1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------------
4
is the stack empty? 0
================================================================================
                        Stack Operations
  1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------------
1
Stack Overflow
```

```
void pop(){
    //error checking
    if(isEmpty()){
        std::cout<<"Stack underflow" <<std::endl;
        return;
    }

    //Display the value that we are going to pop
    std:: cout <<"Popping: " <<stackArr[top] <<std::endl;
    //decrement the top value from the stack
    top--;
}
```

```
================================================================================
                        Stack Operations
  1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exi
--------------------------------------------------------------------------------
2
Popping: 7
--------------------------------------------------------------------------------
```

```
void Top(){
    //error catching:
    if(isEmpty()){
        std::cout<< "the stack is empty" <<std::endl;
        return;
    }
//check the top value

std::cout<<"top element: "<< stackArr[top]<<std::endl;
}
```

```
==========================================================================
                        Stack Operations
   1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------
3
top element: 7
```

```
void displayStack(){
    if (isEmpty()) {
        std::cout << "The stack is empty." << std::endl;
        return;
    }

    std::cout << "Stack elements (top to bottom):" << std::endl;
    for (int i = top; i >= 0; i--) {
        std::cout << stackArr[i] << std::endl;
    }
}
```

```
==========================================================================
                        Stack Operations
   1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------
6
Stack elements (top to bottom):
6
8
4
4
6
0
9
5
6
2
```

```
==========================================================================
                        Stack Operations
   1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------
6
The stack is empty.
```

```
void exit() {
    std::cout << "Exiting the program." << std::endl;
    std::exit(0);
}
```

```
==========================================================================
                        Stack Operations
   1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display the stack  7.Exit
--------------------------------------------------------------------------
7
Exiting the program.
```

ANALYSIS:

In this code, building the code of the functions manually using an array helped me understand what was happening internally in the code itself. It shows the track of the top index myself and checks for overflow and underflow before every push and pop, which the STL normally handles on its own. The isFull() and isEmpty() functions were important since they prevented invalid operations like pushing past maxCap or popping from an empty stack. Adding displayStack() also helped me see the stack elements in order (top to bottom) instead of just the top value, which made testing push and pop easier to verify visually.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C

```cpp
#include <iostream>
#include <string>
#include <iomanip>
#include "stackList.h"

bool isMatching(char openSym, char closeSym);
bool isBalanced(const std::string &expr);

int main() {
    std::string expressions[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B]+[C-D]}"
    };


    std::cout << "---------------------------\n";
    std::cout << std::left << std::setw(18) << "Expression" << "Result\n";

    for (const std::string &expr : expressions) {
        while (!isEmpty()) pop(); // reset stack before checking the next expression

        bool ok = isBalanced(expr);

        std::cout << std::left << std::setw(18) << expr
                  << (ok ? "Valid" : "Invalid") << "\n";
    }

    std::cout << "---------------------------\n";

    return 0;
}

bool isMatching(char openSym, char closeSym) {
    if (openSym == '(' && closeSym == ')') return true;
    if (openSym == '{' && closeSym == '}') return true;
    if (openSym == '[' && closeSym == ']') return true;

    return false;
}

bool isBalanced(const std::string &expr) {
    for (char c : expr) {
        if (c != '(' && c != ')' && c != '{' && c != '}' && c != '[' && c != ']')
            continue;

        if (c == '(' || c == '{' || c == '[') {
            push(c);
            continue;
        }

        if (isEmpty() || !isMatching(pop(), c))
            return false;
    }

    return isEmpty();
}
```

MAIN FILE

```cpp
#include <iostream>
#include <string>

const size_t maxCap = 100;

char stackArr[maxCap];
int top = -1;

void push(char c);
char pop();
bool isEmpty();
bool isMatching(char openSym, char closeSym);
bool isBalanced(const std::string &expr);

int main() {
    std::string expressions[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B]+[C-D])"
    };

    for (int i = 0; i < 4; i++) {
        top = -1; // reset the stack before checking the next expression

        std::cout << "Expression: " << expressions[i] << std::endl;

        if (isBalanced(expressions[i]))
            std::cout << "Valid? Y" << std::endl;
        else
            std::cout << "Valid? N" << std::endl;

        std::cout << std::endl;
    }

    return 0;
}

bool isEmpty() {
    if (top == -1)
        return true;

    return false;
}

void push(char c) {
    if (top == (int)maxCap - 1) {
        std::cout << "Stack Overflow." << std::endl;
        return;
    }

    stackArr[++top] = c;
}

char pop() {
    if (isEmpty()) {
        std::cout << "Stack Underflow." << std::endl;
        return '\0';
    }

    return stackArr[top--];
}
```

```cpp
bool isMatching(char openSym, char closeSym) {
    if (openSym == '(' && closeSym == ')') return true;
    if (openSym == '{' && closeSym == '}') return true;
    if (openSym == '[' && closeSym == ']') return true;

    return false;
}

bool isBalanced(const std::string &expr) {
    for (size_t i = 0; i < expr.length(); i++) {
        char c = expr[i];

        // a) not a symbol to be balanced - ignore it
        if (c != '(' && c != ')' && c != '{' && c != '}' && c != '[' && c != ']')
            continue;

        // b) opening symbol - push it onto the stack
        if (c == '(' || c == '{' || c == '[') {
            push(c);
            continue;
        }

        // c) closing symbol
        // i) report an error if the stack is empty
        if (isEmpty()) {
            std::cout << "Error: '" << c << "' has no matching opening symbol." << std::endl;
            return false;
        }

        // ii) otherwise, pop the stack
        char openSym = pop();

        // d) if the symbol popped does not match, report an error
        if (!isMatching(openSym, c)) {
            std::cout << "Error: '" << openSym << "' does not match '" << c << "'." << std::endl;
            return false;
        }
    }

    // 3) at the end of input, if the stack is not empty: report an error
    if (!isEmpty()) {
        std::cout << "Error: unmatched opening symbol(s) remain." << std::endl;
        return false;
    }

    return true;
}
```

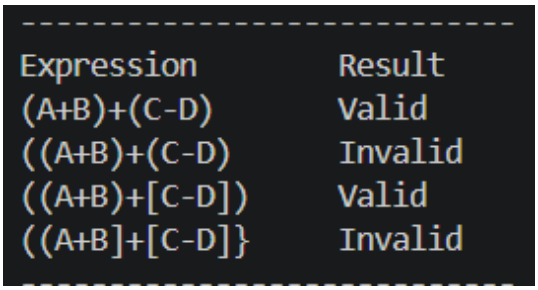
ARRAY FILE

```cpp
#ifndef STACKLIST_H
#define STACKLIST_H

#include <iostream>

class Node {
public:
    char data;
    Node* next;
};

inline Node *head = NULL, *tail = NULL;

inline bool isEmpty() {
    if (head == NULL)
        return true;

    return false;
}

inline void push(char newData) {
    Node *newNode = new Node;

    newNode->data = newData;
    newNode->next = head;

    if (head == NULL) {
        head = tail = newNode;
    } else {
        newNode->next = head;
        head = newNode;
    }
}

inline char pop() {
    char tempVal;
    Node *temp;

    if (isEmpty()) {
        std::cout << "Stack Underflow." << std::endl;
        return '\0';
    }

    temp = head;
    tempVal = temp->data;
    head = head->next;
    delete temp;

    return tempVal;
}

#endif
```

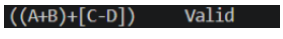
LINK LIST HEADER FILE

COMPUTER ENGINEERING

QUEZON CITY



OUTPUT

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		Every opening parenthesis has a matching closing one in the correct order, so the stack ends up empty by the end of the expression, making it valid.
((A+B)+(C-D)	N		There are three opening parentheses but only two closing ones, so one (is left unmatched in the stack at the end, making it invalid.
((A+B)+[C-D])	Y		Even though (and [are mixed together, each closing symbol correctly matches the last opening symbol pushed, so the stack empties out properly and the expression is valid.

((A+B]+[C-D]}	N	((A+B]+[C-D]} Invalid	The] tries to close a (, and the } tries to close a [, so the symbols don't correspond to each other even though the count of opening and closing symbols matches, making it invalid.

CONCLUSION:

In this activity, it helped me to understand on how a stack works and show how it follows the last in first out order. The stack can be done by using an stl, array or link list, where each one has a different way of managing the elements and checking the overflow and underflow.

In working through ILO A showed me how much the STL simplifies stack operations since push, pop, top, empty, and size are already built in. ILO B gave me a better understanding of what's happening internally, since I had to manually track the top index for the array version and manage node pointers for the linked list version.

The balanced symbols checker in ILO C made the practical use of a stack clearer to me. Since the last opening symbol read has to be the first one closed, a stack fits the problem naturally. Testing it against the four expressions showed how the algorithm catches different kinds of errors, like an opening symbol left unmatched at the end, or a closing symbol that doesn't correspond to the last one opened.

I think in this activity that I learned a lot, my area for improvement is being more careful with small details like missing parentheses in function calls or brace placement, since I ran into a few of these bugs while testing my code and had to go back and fix them especially from the function definition to calling them

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Microsoft copilot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Explanation of concepts, wordings

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Used AI in the google search bar to help me explain my explanation be better like synonyms and how to grammatically say this right.

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.